

FRONTEND INTERVIEW GUIDE - Complete

React, TypeScript, Hooks & State Management *Date: 2026-09-17 | LMS Project*

TABLE OF CONTENTS

1. React Hooks & Patterns
 2. State Management Architecture
 3. TypeScript & Type Safety
 4. API Calling & Authentication
 5. Performance Optimization
 6. OOP & Patterns in JavaScript
 7. Interview Design Questions
 8. Code Examples & Snippets
-

SECTION 1: REACT HOOKS & PATTERNS

useState - Local Component State

Use Case: Form inputs, toggles, UI visibility flags

```
const [activeTab, setActiveTab] = useState<string>('learning');
const [saving, setSaving] = useState(false);
const [formData, setFormData] = useState({});
```

When to use:

- Simple values (string, boolean, number)
 - No interdependencies with other state
 - Component-local state only
-

useEffect - Side Effects & Lifecycle

Use Case: Data fetching, subscriptions, cleanup

```
useEffect(() => {
  const fetchData = async () => {
    const data = await courseService.fetch();
    setData(data);
  };
  fetchData();
}, [userId]); // Dependency array

return () => {
```

```
// Cleanup: unsubscribe, cancel requests
};
```

Common mistakes: ✗ Missing dependency array (runs every render) ✗ Infinite dependencies (function literals in array) ✓ Include all external values used inside effect

useReducer - Complex State Management

Use Case: Multiple related state updates, interdependencies

```
const [state, dispatch] = useReducer(appReducer, initialState);

export const appReducer = (state: AppState, action: any) => {
  switch(action.type) {
    case 'SET_COURSES_AS_FAV':
      return { ...state, favorites: action.payload };
    case 'SET_ACTIVE_TAB':
      return { ...state, activeTab: action.payload };
    default:
      return state;
  }
};
```

Benefits:

- Predictable state transitions
 - Testable reducer logic
 - Easy to debug (single place for logic)
-

useCallback - Memoize Functions

Use Case: Pass functions to optimized child components

```
const handleInputChange = useCallback((content: string) => {
  setFormData(prev => ({...prev, description: content}));
}, []); // No external deps

// Pass to memoized child
<RichTextEditor onChange={handleInputChange} />
```

When beneficial:

- Function passed to memoized child (React.memo)
 - Expensive child re-render without it ✗ DON'T use for every callback (overhead > benefit)
-

useMemo - Memoize Expensive Computation

Use Case: Expensive calculations, complex object creation

```
const filteredCourses = useMemo(() => {  
  return courses.filter(c => c.title.includes(searchQuery))  
    .sort((a, b) => a.rating - b.rating);  
}, [courses, searchQuery]);
```

When beneficial:

- Expensive computation (sorting 10k items)
 - Object passed as dep to child component ❌ DON'T use for simple array operations
-

useRef - Mutable Value That Persists

Use Case: Scroll position, focus, timeout IDs, stable function refs

```
const contentRef = useRef<HTMLDivElement>(null);  
const searchTimeoutRef = useRef<NodeJS.Timeout>();  
  
// Use for direct DOM manipulation  
contentRef.current?.scrollIntoView();  
  
// Use to store function reference  
function debouncedSearch() {  
  clearTimeout(searchTimeoutRef.current);  
  searchTimeoutRef.current = setTimeout(() => {  
    performSearch();  
  }, 300);  
}
```

useContext - Global State Access

Use Case: Avoid prop drilling across many levels

```
export const useAppContext = () => {  
  const context = useContext(AppContext);  
  if (!context) throw new Error("Must be within AppContextProvider");  
  return context;  
};  
  
// In any component  
const { state, dispatch } = useAppContext();
```

SECTION 2: STATE MANAGEMENT ARCHITECTURE

Context API + useReducer Pattern

This system uses Context API (NO Redux) for global state:

```
// 1. Create Context
export const AppContext = createContext<{
  state: AppState;
  dispatch: Dispatch<any>;
} | undefined>(undefined);

// 2. Provider Component
export const AppContextProvider: React.FC = ({ children }) => {
  const [state, dispatch] = useReducer(appReducer, initialState);

  useEffect(() => {
    // On mount: fetch landing page data
    Promise.allSettled([
      fetchEnrolledEvents(dispatch, user.userId),
      fetchUpcomingEvents(dispatch, user.userId),
      fetchAssemblyCount(dispatch, user.userId)
    ]).then(() => dispatch(setLandingDataLoaded()));
  }, [user.userId]);

  return (
    <AppContext.Provider value={{ state, dispatch }}>
      {children}
    </AppContext.Provider>
  );
};

// 3. Custom Hook for Consumption
export const useAppContext = () => {
  const context = useContext(AppContext);
  if (!context) throw new Error("useAppContext outside provider");
  return context;
};
```

AppState Interface - Comprehensive

```
export interface AppState {
  // Data
  favorites: { id: string; isFavorite: boolean }[];
  enrolledCourses: any[];
  enrolledEvents: any[];
  enrolledAssemblies: Record<string, AssemblyEnrollmentSummary>;
  upcomingEventsDashboard: any[];
  assemblyCount: { dueCount?: number; inProgressCount?: number };
  profileDetails: UserProfileDetails | null;
}
```

```

    acquiredSkills: Skill[];
    edsSkills: EDSSkill[];

    // Metadata
    searchQuery: string;
    searchContext: SearchContextSnapshot | null;
    launchLinks: { [lessonId: string]: string };

    // Fetch Status Flags
    enrolledCoursesFetched: boolean;
    enrolledEventsFetched: boolean;
    profileDetailsFetched: boolean;
    landingDataLoaded: boolean;

    // Mutations
    continueLearningAndFavs: any | null;
    continueLearningAndFavsDirty: boolean;
  }

```

Action Pattern - Dispatchers

```

// In reducer
export const setCourseAsFav = (id: string, isFavorite: boolean) => ({
  type: 'SET_COURSE_AS_FAV',
  payload: { id, isFavorite }
});

// Async action dispatcher (outside reducer)
export const fetchAndSetEnrolledCourses = async (
  dispatch: Dispatch<any>,
  userId: string
) => {
  try {
    const response = await enrollmentService.getEnrolledCourses(userId);
    dispatch(setEnrolledCourses(response?.data?.courses || []));
  } catch (error) {
    console.error("Error fetching enrollments:", error);
    dispatch(setEnrolledCourses([])); // Fail gracefully
  }
};

// Usage in component
const handleEnroll = async () => {
  await fetchAndSetEnrolledCourses(dispatch, userId);
};

```

Why NOT Redux?

Aspect	Context API	Redux
--------	-------------	-------

Aspect	Context API	Redux
Boilerplate	Minimal	Extensive (actions, reducers, selectors)
Learning Curve	Easy	Steep
Debugg	CloudWatch/manual	Redux DevTools
Async Logic	Custom	Middleware (thunk, saga)
Best For	Mid-sized apps	Large complex apps

This system's choice: Context because:

- ~30 action types only
- Mid-sized app, fast iteration priority
- No complex async workflows needing saga

SECTION 3: TYPESCRIPT & TYPE SAFETY

Discriminated Unions for Type Narrowing

```
interface RadioButtonQuestion extends BaseQuestion {
  questionType: "radio";
  options: Option[];
}

interface CheckboxQuestion extends BaseQuestion {
  questionType: "checkbox";
  options: Option[];
}

interface LikertScaleQuestion extends BaseQuestion {
  questionType: "likertScale";
  scalePath: string[];
}

type SurveyQuestion = RadioButtonQuestion | CheckboxQuestion |
LikertScaleQuestion;

// Type guard using discriminant
function renderQuestion(q: SurveyQuestion) {
  if (q.questionType === "radio") {
    // TypeScript knows q is RadioButtonQuestion here
    return q.options.map(opt => <Radio key={opt.id} label={opt.label} />);
  } else if (q.questionType === "checkbox") {
    // TypeScript knows q is CheckboxQuestion
    return q.options.map(opt => <Checkbox key={opt.id} label={opt.label} />);
  }
  // More cases...
}
```

Benefits:

- Compile-time safety (no runtime checks)
 - Autocomplete works for each variant
 - Exhaustiveness checking (forget a case = error)
-

Generic Types for Reusable Components

```
export function useWindowedPagination<T>({
  fetchPage,
  resetToken = "",
  initialRowsPerPage = 10,
  prefetchPagesAhead = 2,
  enabled = true,
}: UseWindowedPaginationOptions<T>) {
  const [items, setItems] = useState<T[]>([]);
  const [nextKey, setNextKey] = useState<string | null>(null);

  // ... implementation

  return { items: T[], pageItems: T[], loading: boolean };
}

// Usage - Type-safe for any entity
const { items: courses } = useWindowedPagination<Course>({
  fetchPage: (cursor) => courseService.fetch(cursor)
});

const { items: users } = useWindowedPagination<User>({
  fetchPage: (cursor) => userService.fetch(cursor)
});

// Autocomplete: courses[0]. → CourseID, title, etc.
```

Callback & Event Types

```
export type TokenCallback = () => { idToken: string };
export type SetTokensCallback = (tokens: {
  idToken: string;
  tokenExpiresIn?: number;
  tokenGeneratedAt?: Date;
}) => void;
export type RefreshTokensCallback = (forceRefresh?: boolean) => Promise<boolean>;
```

SECTION 4: API CALLING & AUTHENTICATION

fetchWithAuth - Centralized Auth Logic

```
export const fetchWithAuth = async (
  url: string,
  options: RequestInit = {}
) => {
  const headers = getAuthHeaders();
  return await callAPI(url, headers, options);
};

export const getAuthHeaders = () => {
  const tenant = window.location.host;
  return new Headers({
    [headersTennant.TENANT_HEADER]: tenant,
    Authorization: `Bearer ${getToken().idToken}`,
  });
};

// With token refresh on 401
let refreshPromise: Promise<boolean> | null = null;

if (response.status === 401 || response.status === 403) {
  if (!refreshPromise) {
    refreshPromise = callShellRefresh().finally(() => {
      refreshPromise = null;
    });
  }
  const refreshed = await refreshPromise;
  if (refreshed) {
    // Retry original request with new token
    return fetch(url, {
      ...options,
      headers: getAuthHeaders() // Fresh headers
    });
  }
}
```

Service Layer Architecture

Each domain has its own service file:

- assemblyService.ts - Assembly operations
- courseService.ts - Course enrollment & details
- userService.ts - User profile, search
- eventService.ts - Event management
- etc. (29 files in Admin UI)

```
// assemblyService.ts
const BASE_URL = `${process.env.NEXT_PUBLIC_COURSE_API}/api/v1/assemblies`;
```

```
export const retrieveUserEnrolledAssemblies = async (userId: string) => {
  const url = `${BASE_URL}/${userId}/enrolled-structures`;
  return await fetchWithAuth(url);
};

export const fetchFavoriteAssemblies = async (userId: string) => {
  return await fetchWithAuth(`${BASE_URL}/${userId}/favourite-assemblies`);
};
```

Auth Variants

```
// Standard REST
export const fetchWithAuth = async (url, options) => {
  headers: getAuthHeaders() // Bearer token + tenant
};

// GenAI endpoints (different refresh logic)
export const fetchWithGenAIAuth = async (url, options) => {
  // Uses genAI-specific token + refresh
};

// Rustici SCORM integration
export const fetchWithAuthRustici = async (url, options) => {
  // Custom headers for SCORM compliance
};

// KnowledgeX third-party API
export const fetchWithAuthKnowledgeX = async (url, options) => {
  // Tenant slug in URL path
};
```

SECTION 5: PERFORMANCE OPTIMIZATION

Custom Hook: useWindowedPagination

Implements cursor-based pagination with client-side windowing:

```
export function useWindowedPagination<T>({
  fetchPage, // (cursor: string) => Promise<WindowedPage<T>>
  resetToken = "",
  initialRowsPerPage = 10,
  prefetchPagesAhead = 2,
  enabled = true,
}: UseWindowedPaginationOptions<T>) {
  const [items, setItems] = useState<T[]>([]);
  const [nextKey, setNextKey] = useState<string | null>(null);
  const [loading, setLoading] = useState(false);
```

```

const fetchPageRef = useRef(fetchPage);
const requestIdRef = useRef(0); // Cancel stale requests

const load = useCallback(async (silent: boolean) => {
  const reqId = ++requestIdRef.current;
  setLoading(!silent);

  try {
    const cursor = nextKey || "";
    const result = await fetchPageRef.current(cursor);

    // Ignore stale responses
    if (reqId !== requestIdRef.current) return;

    setItems(prev => [...prev, ...result.items]);
    setNextKey(result.nextCursor);
  } finally {
    setLoading(false);
  }
}, [nextKey]);

// Auto-prefetch next batch as user scrolls
useEffect(() => {
  if (!enabled || !nextKey || loadingMore) return;

  const threshold = (page + prefetchPagesAhead) * rowsPerPage;
  if (threshold >= items.length) {
    loadMore(); // Fetch next batch
  }
}, [page, rowsPerPage, items.length, nextKey]);

return {
  items,
  pageItems: items.slice(page * rowsPerPage, (page + 1) * rowsPerPage),
  page,
  setPage,
  total: items.length,
  loading,
  loadMore: () => load(false)
};
}

```

Code Splitting with React.lazy

```

// Lazy load heavy components
const UserManagement = React.lazy(() =>
  import("@/components/UserManagement")
);

// Render with Suspense fallback
<Suspense fallback={<CircularProgress />>

```

```

<Routes>
  <Route path="/admin/users" element={<UserManagement />} />
</Routes>
</Suspense>

```

Benefits:

- Initial bundle smaller (smaller first load)
- Only admin users download UserManagement code
- Fallback shown while chunk loads (100-500ms)

Trade-off:

- Network request added (latency for first access)
- Use for heavy components, not small utility components

SECTION 6: OOP & PATTERNS IN JAVASCRIPT

Static Utility Classes

```

export class CDataCollector {
  static browserName(ua?: string): string {
    const userAgent = ua ?? navigator.userAgent;
    if (/Edg\\/.test(userAgent)) return "Edge";
    if (/Chrome\\/.test(userAgent)) return "Chrome";
    if (/Firefox\\/.test(userAgent)) return "Firefox";
    if (/Safari\\/.test(userAgent)) return "Safari";
    return "Unknown";
  }

  static deviceType(ua?: string): "mobile" | "tablet" | "desktop" | "unknown" {
    const userAgent = ua ?? navigator.userAgent;
    if (/Mobi|Android/.test(userAgent)) return "mobile";
    if (/Tablet|iPad/.test(userAgent)) return "tablet";
    return "desktop";
  }

  static async publicIp(): Promise<string> {
    try {
      const res = await fetch("https://api.ipify.org?format=json");
      const data = await res.json() as { ip?: string };
      return data.ip ?? "unknown";
    } catch {
      return "unknown";
    }
  }

  static async buildAsync(): Promise<CDataItem[]> {
    return [
      { id: this.getSessionId(), type: "UserSession" }
    ]
  }
}

```

```

    ];
  }
}

// Usage: No instantiation needed
CDataCollector.browserName() // "Chrome"

```

Event Bubbling & Delegation

```

// Event bubbling: events propagate UP the DOM
<div onClick={handleDivClick}>      {/* Triggered 3rd */}
  <button onClick={handleButtonClick}> {/* Triggered 1st */}
    Click
  </button>
  <span onClick={handleSpanClick} />  {/* Triggered 2nd */}
</div>

// Event delegation: Handle parent, manage child events
<div onClick={(e) => {
  if (e.target.tagName === "BUTTON") {
    handleButtonClick(e);
  }
}}>
  {/* Many buttons here - single handler for all */}
  {buttons.map(btn => <button>{btn}</button>)}
</div>

```

Benefits:

- Single handler for many elements
- Handles dynamically added elements
- Memory efficient (1 listener vs 100)

SECTION 7: INTERVIEW DESIGN QUESTIONS

Q: Design an API calling service. What considerations matter?

Requirements:

1. Authentication: Inject JWT, handle refresh
2. Error handling: Retry on 5xx, fail on 4xx
3. Timeout: Cancel after 30s
4. Caching: Cache GET with TTL
5. Monitoring: Track response times, errors
6. Type safety: Generic types

Implementation:

```
export class ApiClient {
  private requestInterceptor = (c) => c;
  private responseInterceptor = (r) => r;

  async get<T>(url: string, opts?: RequestOpts): Promise<T> {
    const cacheKey = `GET:${url}`;
    const cached = getCache<T>(cacheKey, opts?.cacheTtl);
    if (cached) return cached;

    const controller = new AbortController();
    const timeout = setTimeout(
      () => controller.abort(),
      opts?.timeout ?? 30000
    );

    try {
      const response = await fetch(url, {
        headers: this.requestInterceptor(opts?.headers ?? {}),
        signal: controller.signal
      });
      const data = await response.json();
      setCache(cacheKey, data);
      return this.responseInterceptor(data);
    } finally {
      clearTimeout(timeout);
    }
  }
}
```

Q: What happens between user click and data displayed?

Flow: Search for courses

1. User types in SearchBox, onChange fires
2. debounce(300ms) waits for typing stop
3. Dispatch to AppContext: setSearchQuery(query)
4. AppContext reducer updates state
5. Component re-renders with new query
6. Call courseService.search(query)
7. fetchWithAuth adds token + tenant header
8. API Gateway + Lambda Authorizer validates
9. Backend queries DynamoDB
10. Results returned
11. Dispatch: setSearchResults(data)
12. Component re-renders with results
13. Results cached in localStorage (5-min TTL)
14. User sees results + clicks to enroll

Performance considerations:

- Caching: Avoid repeated searches
- Error handling: Show error message + retry button
- Loading UI: Skeleton or spinner shown

SECTION 8: LOCAL & SESSION STORAGE

TTL-Aware Storage Wrapper

```
interface CacheEnvelope<T> {
  v: T; // Value
  t: number; // Timestamp
}

export const getCached = <T>(key: string, ttlMs?: number): T | null => {
  if (typeof window === "undefined") return null;
  try {
    const raw = window.localStorage.getItem(key);
    if (!raw) return null;

    const parsed = JSON.parse(raw) as CacheEnvelope<T>;

    // Check if expired
    if (ttlMs && parsed?.t && Date.now() - parsed.t > ttlMs) {
      return null;
    }

    return parsed?.v ?? null;
  } catch {
    return null;
  }
};

export const setCached = <T>(key: string, value: T): void => {
  if (typeof window === "undefined") return;
  try {
    window.localStorage.setItem(
      key,
      JSON.stringify({ v: value, t: Date.now() })
    );
  } catch {
    // Quota exceeded - ignore silently
  }
};

export const removeCachedByPrefix = (prefix: string, keepKey?: string) => {
  const toRemove: string[] = [];
  for (let i = 0; i < window.localStorage.length; i++) {
    const key = window.localStorage.key(i);
    if (key && key.startsWith(prefix) && key !== keepKey) {
      toRemove.push(key);
    }
  }
}
```

```
    }  
    toRemove.forEach(key => window.localStorage.removeItem(key));  
  };  
};
```

localStorage vs sessionStorage

Feature	localStorage	sessionStorage
Persistence	After browser close	Cleared on tab close
Use Case	User prefs, cached data	Temporary state during nav
Storage	~5-10MB per domain	~5MB per tab
This System	User profile, courses	Temporary selections

NPM PACKAGES USED

Category	Packages
UI Framework	@mui/material v6, @mui/x-data-grid v7
State	react-router-dom v7, react-i18next v16
Rich Text	@tiptap/core v3, @tiptap/react v3, mui-tiptap v1
Drag/Drop	@dnd-kit/core v6, @dnd-kit/sortable v10
Forms	react-dropzone v14, react-easy-crop v5
AWS SDK	@aws-sdk/client-bedrock-runtime v3
Utils	dayjs v1, uuid v14, dompurify v3
Document	pdfjs-dist v4, mammoth v1, jszip v3

KEY INTERVIEW TIPS

1. **Explain the WHY:** Not just what technology, but why chosen
2. **Walk through flows:** Be ready to diagram request paths
3. **Know tradeoffs:** Context vs Redux, useState vs useReducer
4. **Think about scale:** How would this handle 10x users?
5. **Security awareness:** Token refresh, tenant isolation
6. **Performance focus:** Memoization, code splitting, caching
7. **Error handling:** What happens when API fails? Token expires?
8. **Testing mindset:** How would you test these components?

Interview Prep Complete - Study this with the Backend and AWS guides for comprehensive coverage